

9주차 2차시

소프트웨어 공학(2)

소프트웨어 생산의 경제 원리_소프트웨어를 효율적으로 개발·운영하는 방법

- 1 소프트웨어 유지 보수
- 2 소프트웨어 품질관리
- 3 소프트웨어 공학의 발전 동향



소프트웨어 공학(2)

◆ 학습목표

- 소프트웨어 유지 보수 절차와 품질관리 유형을 알아본다.
- 소프트웨어 공학의 발전 동향을 살펴본다.

1. 소프트웨어 유지 보수

◆ 유지 보수 절차

- 분석
- 설계
- 구현
- 테스트

◆ 유지 보수 절차



〈출처 : B. Boehm and V. Basili, "Software Defect Reduction Top 10 List", IEEE Computer〉

2.

소프트웨어 품질관리

1) 소프트웨어 품질 관리

◆ 소프트웨어 제품 품질

- 소프트웨어 제품 품질은 국제표준인 ISO/IEC9126에서 정한 정의와 기준에 따라 측정되고 평가된다.
 - ① 기능성 : 요구사항이 무엇인지 식별하고 제품에 요구된 기능과 실제제품이 정확히 일치하는지 측정한다.
 - ② 신뢰성: 수명주기 중 고장이 날때까지의 평균 기간을 의미하며 MTBF라고도 부른다.
 - ③ 사용 용이성: 사용자가 얼마나 편리하게 사용할 수 있는가를 나타낸다.
 - ④ 효율성: 비용대비 효과 차원에서 얼마나 빠른가를 나타낸다.
 - ⑤ 유지 보수성: 고장이 난 후 수리 기간이 얼마나 걸리는 지를 나타낸다.
 - ⑥ 이식성: 해당 소프트웨어를 다른 시스템에서 동작 시켰을때 문제없이 돌아갈 수 있는지를 나타낸다.

2. 소프트웨어 품질관리

2) 소프트웨어 프로세스 품질

◆ SPICE

- 아시아, 유럽, 호주 등에서 주로 사용하는 프로세스 품질평가표준으로 1995년에 ISO에서 제정하였다. 총 40개의 프로세스를 대상으로 성숙도를 평가하고 미비점은 성능 개선을 촉구한다.

표 8-5 SPICE 성숙도 단계별 수준

단계	수준	설명
5	최적화(optimizing)	현재는 물론 미래 프로세스에 적용해도 목표를 만족시키며 지속적으로 개선할 수 있는 수준
4	예측(predictable)	표준 프로세스가 일관된 통제에 따라 수행되며 정량적으로 성과를 측정할 수 있는 수준
3	확립(established)	표준으로 정해진 프로세스에 따라 결과가 산출되는 수준
2	관리(managed)	정해진 절차 및 계획에 따라 결과가 산출되는 수준
1	비정형적 실행(performed)	계획에 따라 꾸준히 결과물이 산출되는 것이 아니라 일시적으로 산출되는 수준
0	불안정(incomplete)	작업 산출물이나 결과를 기대하기 어려운 수준

2. 소프트웨어 품질관리

2) 소프트웨어 프로세스 품질

◆ CMM

- 1992년에 미국 국방성의 지원을 받아 만들어진 모델이다. SPICE와 유사하나 1단계부터 5단계까지 다섯 단계로 나뉘 평가된다. 미국 국방성에서는 CMM에 따라 평가 받은 결과를 토대로 입찰업체를 선정한다. 따라서 세계 유수의 업체가 관심을 가지고 참여하고 있다.

3. 소프트웨어 공학의 발전 동향

1) 프로그램 테스트

◆ 개념

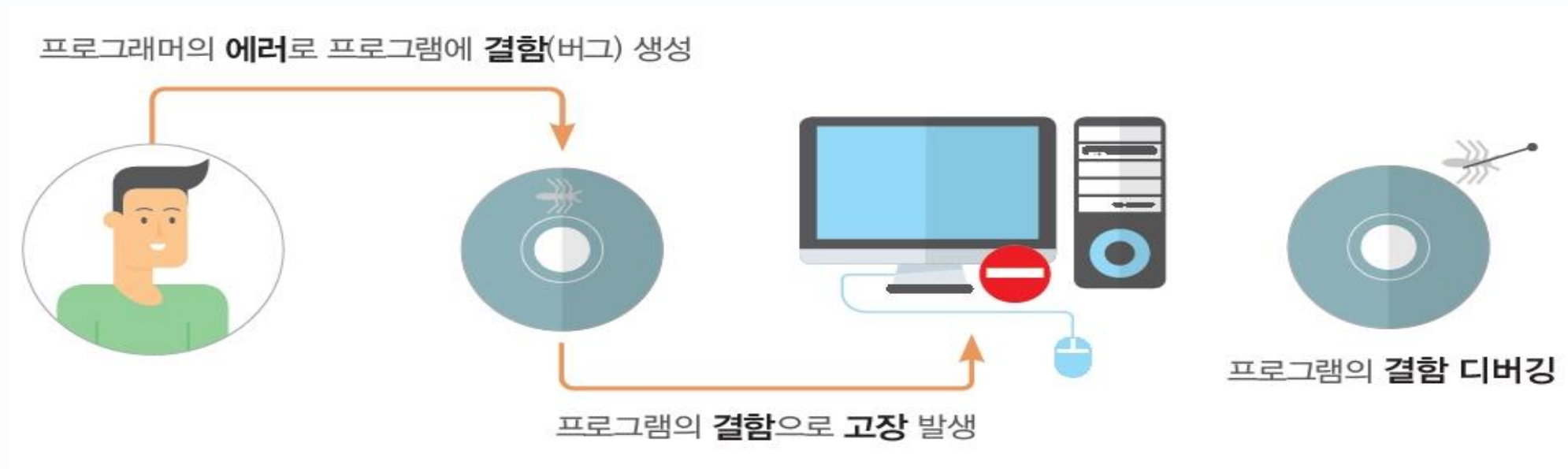
- 프로그램 실행을 통하여 프로그램에 존재하는 결함을 검출하는 작업
- 개발된 프로그램이 요구사항을 모두 만족하는지 확인하는 작업

3. 소프트웨어 공학의 발전 동향

1) 프로그램 테스트

◆ 에러, 결함, 고장, 디버깅

- 에러: 프로그래머가 프로그래밍할 때 범하는 실수, 프로그램 내 결함 유발
- 결함: 프로그램 내에 존재하는 불완전, 올바르지 않은 부분, 흔히 버그라고도 함
- 고장: 프로그램 내 존재하는 결함으로 인해 발생하는 프로그램의 잘못된 행동
- 디버깅: 프로그램 내의 결함을 찾아내어 제거하는 작업



3. 소프트웨어 공학의 발전 동향

2) 프로그램 테스트의 개념과 예

◆ 테스트 방식

- 블랙박스 테스트: 프로그램 내용을 보지 않고 주어진 테스트 시나리오대로 실행 시키면서 오류를 검사
- 화이트박스 테스트: 프로그램 내용을 직접 보면서 프로그램의 오류를 검사

◆ 테스트 종류

구분	설명	테스트 방식
시스템 테스트	프로그램을 실제로 실행하면서 결함을 검출	블랙박스 테스트
단위 테스트	프로시저 단위로 결함 검출	화이트박스 테스트
결합 테스트	프로시저들이 정상적으로 하나의 프로그램으로 결합되는지 검사	화이트박스 테스트

3. 소프트웨어 공학의 발전 동향

3) 테스트케이스의 개념과 예

◆ 테스트 케이스란?

- 블랙박스 테스트에 사용되는 하나의 테스트 시나리오

◆ 테스트케이스의 구성요소

- 테스트케이스의 고유 식별자
- 테스트 내용의 개요
- 테스트 범주 구분: 기능, 성능, 안정성, 기타제약등
- 심각성 구분: 매우 심각, 심각, 보통, 사소 등
- 사전조건: 테스트케이스를 실행 전 프로그램이 사전에 만족해야 할 조건
- 프로그램 조작 시나리오

3. 소프트웨어 공학의 발전 동향

3) 테스트케이스의 개념과 예

◆ 테스트케이스 설계의 중요성

- 시스템 테스트는 테스트케이스의 효율성에 크게 영향을 받음
- 예를 들어, 전자계산기 테스트에서 양수끼리 더하는 테스트케이스를 100개 만들어 검사해도 음수끼리 더하는 기능의 결함을 찾을수 없음
- 따라서, 적은 텍스트케이스들로 최대한 많은 결함을 찾도록 테스트케이스 설계

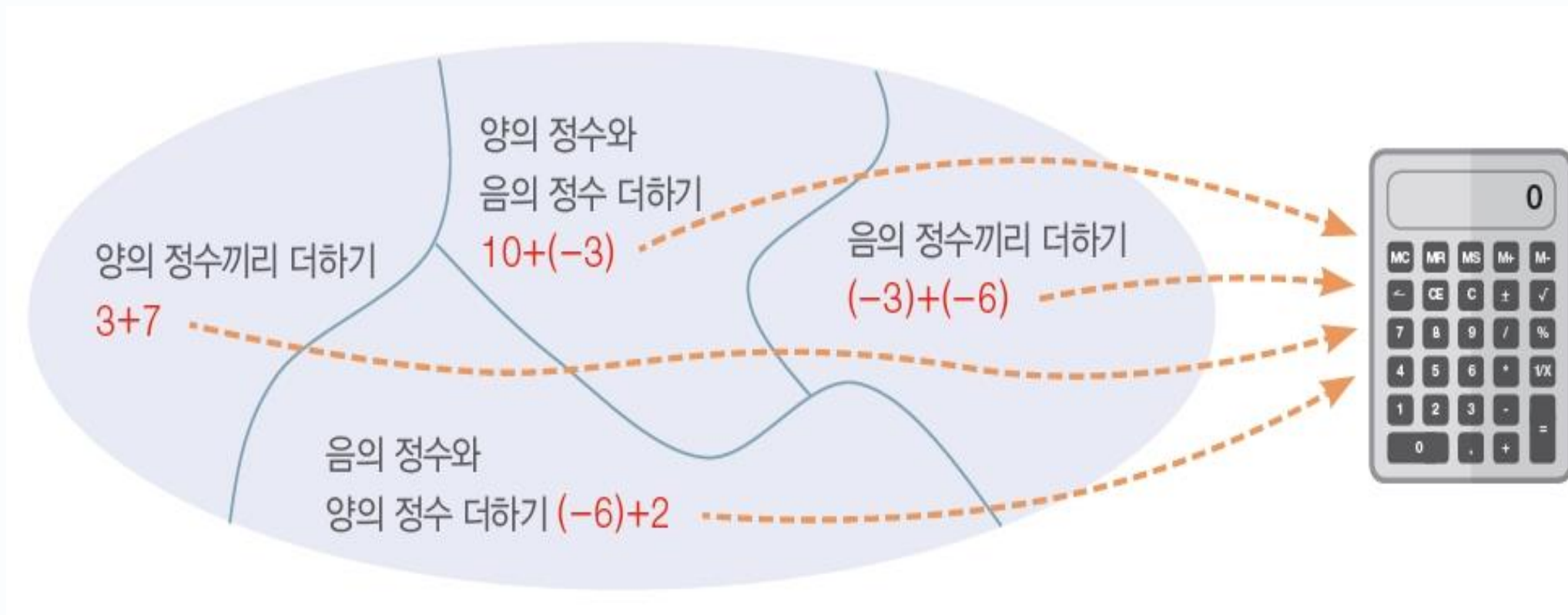


3. 소프트웨어 공학의 발전 동향

3) 테스트케이스의 개념과 예

◆ 테스트케이스 설계 기법

- 입력의 유형별 대표 입력 값 사용(동등 분할 기법)
 - 예를 들어, 전자계산기의 정수 더하기 기능을 검사하는 테스트케이스들을 설계할때, 양수+양수, 양수+음수, 음수+양수, 음수+음수로 유형을 구분하고 각 유형의 대표 입력값으로 테스트케이스 설계

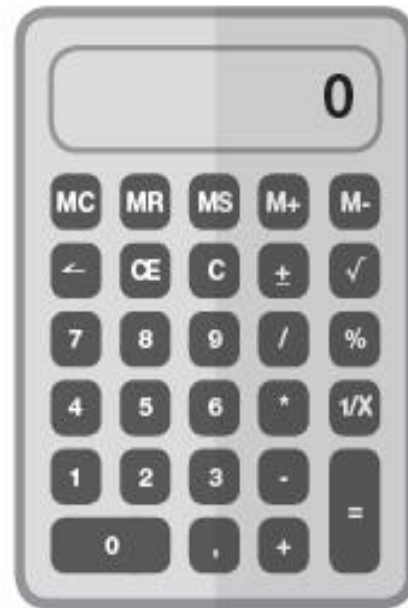


3. 소프트웨어 공학의 발전 동향

3) 테스트케이스의 개념과 예

◆ 테스트케이스 설계 기법

- 의미 있는 경계 값 사용(경계값 분석 기법)
 - 예를 들어, 전자계산기가 계산할 수 있는 최대 값과 최소 값, 그리고 0으로 나누기 등의 경계 값 사용



최대 1억까지 계산할 수 있는 전자계산기

경계값을 활용한 테스트케이스 설계

- 계산 결과가 1억을 넘어가는 테스트케이스, 예) 6천만 + 6천만
- 0으로 나누기 기능을 검사하는 테스트케이스, 예) 25 / 0

3. 소프트웨어 공학의 발전 동향

4) 요구사항 분석과 테스트 연습

◆ 삼각형 종류 구별하기 프로그램의 요구사항 분석

- 프로그램 소개
 - 사용자로부터 삼각형 세 변의 길이를 입력 받음
 - 입력 받은 세 변의 길이가 삼각형을 이루는지 검사
 - 입력 받은 세 변의 길이가 삼각형을 이루면, 정삼각형인지, 정삼각형이 아닌 이등변 삼각형인지, 부등변 삼각형인지 구별

3. 소프트웨어 공학의 발전 동향

4) 요구사항 분석과 테스트 연습

◆ 삼각형 종류 구별하기 프로그램의 요구사항 분석

- 기능에 대한 요구사항
 - 프로그램의 성격 상 성능과 안정성에 대한 요구 사항은 필요하지 않음

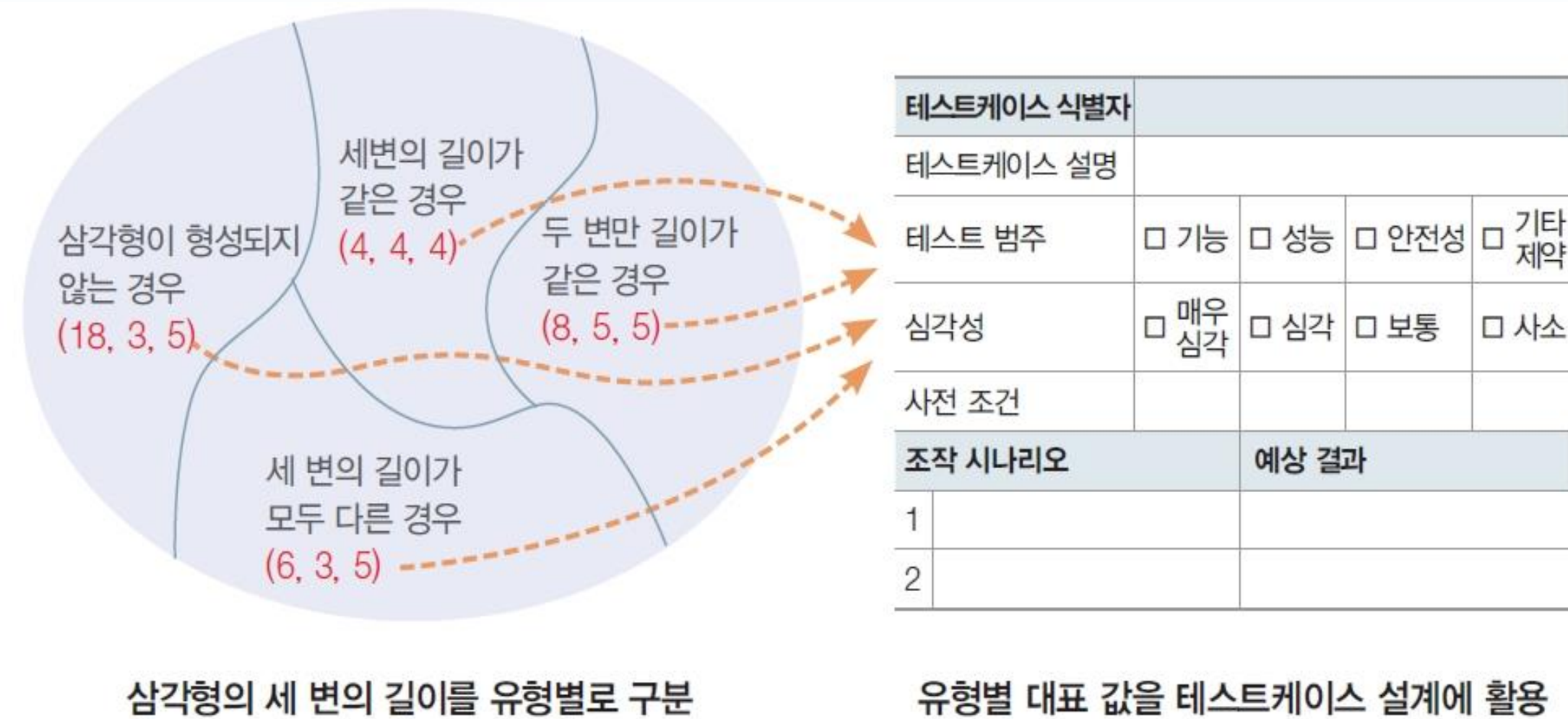
식별자	기능 내용
기능 요구 1	입력받은 세 변의 길이가 삼각형을 형성하지 못하는 값이라면 고양이가 “잘못 된 값입니다.”라고 말합니다.
기능 요구 2	삼각형을 형성하는 세 변의 길이가 모두 같으면 고양이가 “정삼각형입니다.”라고 말합니다.
기능 요구 3	삼각형을 형성하는 세 변의 길이 중 두 변만 길이가 같으면 고양이가 “정삼각형이 아닌 이등변삼각형입니다.”라고 말합니다.
기능 요구 4	삼각형을 형성하는 세 변의 길이가 모두 다르면 고양이가 “부등변삼각형입니다.”라고 말합니다.

3. 소프트웨어 공학의 발전 동향

4) 요구사항 분석과 테스트 연습

◆ 삼각형 종류 구별하기 프로그램의 테스트케이스 설계

- 삼각형 세 변의 유형별 대표 입력 값 사용



3. 소프트웨어 공학의 발전 동향

4) 요구사항 분석과 테스트링 연습

◆ 삼각형 종류 구별하기 프로그램의 테스트케이스 설계

- 기능요구1과 기능요구2의 테스트케이스

테스트케이스 식별자		기능 요구 1 – 테스트케이스 001
테스트케이스 설명		세 변의 길이가 삼각형을 형성하지 못하는 값이라면 고양이가 “잘못된 값입니다.”라고 말하는지 테스트합니다.
조작 시나리오		예상 결과
1	스크래치 프로그램을 실행시킵니다.	사용자 입력 창이 뜨고 고양이가 첫 번째 변의 길이를 묻습니다.
2	사용자 입력 창에 18을 입력합니다	사용자 입력 창이 뜨고 고양이가 두 번째 변의 길이를 묻습니다.
3	사용자 입력 창에 3을 입력합니다.	사용자 입력 창이 뜨고 고양이가 세 번째 변의 길이를 묻습니다.
4	사용자 입력 창에 5를 입력합니다.	고양이가 “잘못된 값입니다.”라고 말합니다.

테스트케이스 식별자		기능 요구 2 – 테스트케이스 001
테스트케이스 설명		세 변의 길이가 모두 같으면 고양이가 “정삼각형입니다.”라고 말하는지 테스트합니다.
조작 시나리오		예상 결과
1	스크래치 프로그램을 실행시킵니다.	사용자 입력 창이 뜨고 고양이가 첫 번째 변의 길이를 묻습니다.
2	사용자 입력 창에 4를 입력합니다.	사용자 입력 창이 뜨고 고양이가 두 번째 변의 길이를 묻습니다.
3	사용자 입력 창에 4를 입력합니다.	사용자 입력 창이 뜨고 고양이가 세 번째 변의 길이를 묻습니다.
4	사용자 입력 창에 4를 입력합니다.	고양이가 “정삼각형입니다.”라고 말합니다.

3. 소프트웨어 공학의 발전 동향

4) 요구사항 분석과 테스트 연습

◆ 삼각형 종류 구별하기 프로그램의 테스트케이스 설계

- 기능요구3과 기능요구4의 테스트케이스

테스트케이스 식별자		기능 요구 3 – 테스트케이스 001
테스트케이스 설명		세 변의 길이 중 두 변만 길이가 같으면 고양이가 “정삼각형이 아닌 이등변삼각형입니다.”라고 말하는지 테스트합니다.
조작 시나리오		예상 결과
1	스크래치 프로그램을 실행시킵니다.	사용자 입력 창이 뜨고 고양이가 첫 번째 변의 길이를 묻습니다.
2	사용자 입력 창에 8을 입력합니다	사용자 입력 창이 뜨고 고양이가 두 번째 변의 길이를 묻습니다.
3	사용자 입력 창에 5를 입력합니다.	사용자 입력 창이 뜨고 고양이가 세 번째 변의 길이를 묻습니다.
4	사용자 입력 창에 5를 입력합니다.	고양이가 “정삼각형이 아닌 이등변삼각형입니다.”라고 말합니다.

테스트케이스 식별자		기능 요구 4 – 테스트케이스 001
테스트케이스 설명		세 변의 길이가 모두 다르면 고양이가 “부등변삼각형입니다.”라고 말하는지 테스트합니다.
조작 시나리오		예상 결과
1	스크래치 프로그램을 실행시킵니다.	사용자 입력 창이 뜨고 고양이가 첫 번째 변의 길이를 묻습니다.
2	사용자 입력 창에 6을 입력합니다.	사용자 입력 창이 뜨고 고양이가 두 번째 변의 길이를 묻습니다.
3	사용자 입력 창에 3을 입력합니다.	사용자 입력 창이 뜨고 고양이가 세 번째 변의 길이를 묻습니다.
4	사용자 입력 창에 5를 입력합니다.	고양이가 “부등변삼각형입니다.”라고 말합니다.

3. 소프트웨어 공학의 발전 동향

5) 웹 엔지니어링

◆ 웹 기반 소프트웨어와 일반 소프트웨어와의 차이

- 네트워크 필수
 - 웹 기반 소프트웨어는 네트워크가 필수이다. 인터넷과 인트라넷을 막론하고 다양한 클라이언트에 대해 서비스를 제공할 수 있어야함
- 콘텐츠 중심
 - 웹 기반 소프트웨어는 하이퍼 미디어를 사용하는 텍스트, 그래픽, 오디오, 비디오 등 다양한 멀티미디어 서비스를 제공함
- 지속적 진화
 - 웹 기반 소프트웨어는 일반 소프트웨어에 비해 빠르게 진화함

3. 소프트웨어 공학의 발전 동향

6) 관점 지향 프로그래밍

- ◆ 소프트웨어 구조와 구현내용이 갈수록 복잡해지고 다차원적으로 변하고 있음
- ◆ 절차 지향 프로그래밍에서는 분해된 관심을 프로시저로 구성했고, 객체 지향 프로그래밍에서는 클래스로 구성
- ◆ 객체 지향 프로그래밍에서 충분히 분리해낼 수 없는 관심이 존재한다는 문제에서 출발

표 8-6 관점 지향 프로그래밍과 객체 지향 프로그래밍의 비교

항목	관점 지향 프로그래밍	객체 지향 프로그래밍
목적	부문별 조립을 통한 생산성 제고	상속 및 재사용을 통한 효율성 제고
산출물	관점	클래스
장점	환경 독립성, 횡단 관심 공통 적용	객체 재활용
단점	구현하고 이해하는 데 오랜 시간 소요	객체별 독립성 유지가 어려움

3. 소프트웨어 공학의 발전 동향

7) 컴포넌트 기반 소프트웨어 개발

- ◆ 소프트웨어도 하드웨어처럼 교체가 필요한 부분을 갈아 끼우면 바로 동작하는 모델
- ◆ 컴포넌트의 크기와 적용 분야에 따른 분류
 - 사용자 인터페이스 컴포넌트
 - 작은 규모라 재사용 빈도는 가장 높지만 다른 컴포넌트에 비해 재사용 했을 때 효용 가치는 떨어짐
 - 기술 기반 컴포넌트
 - 소프트웨어의 종류와 상관없이 다양한 분야에서 사용되는 공통의 컴포넌트이다. 어떤 구체적인 비즈니스논리나 프로세스를 포함하기보다 공통 기술적 서비스 관련 내용을 담음
 - 예외처리 컴포넌트, 보안 컴포넌트, 에러처리 컴포넌트, 트랙잭션 컴포넌트 등이 있음

3. 소프트웨어 공학의 발전 동향

7) 컴포넌트 기반 소프트웨어 개발

◆ 컴포넌트의 크기와 적용 분야에 따른 분류

- 비즈니스 기반 컴포넌트
 - 비즈니스 영역에 상관없이 공통적으로 사용되는 비즈니스 객체나 프로세스를 컴포넌트화한 것임
 - 날짜 변경 컴포넌트, 통화 변경 컴포넌트 등이 있음
- 비즈니스 컴포넌트
 - 특정 영역에 속한 비즈니스에 대한 컴포넌트로 사원, 주문, 급여, 임대 등과 같이 구체적인 비즈니스를 지원함
- 애플리케이션 컴포넌트
 - 특정 애플리케이션에 필요한 사용자 인터 페이스와 비즈니스 제어 흐름을 함께 정의한 컴포넌트
 - 주문처리 컴퍼넌트, 고객 관리 컴포넌트 등이 있음



요약

◆ 소프트웨어 유지보수

- 소프트웨어 개발 비용이 30% 수준인데 비해 유지보수 비용은 70%를 넘는 만큼 전체 비용 측면에서 소프트웨어 유지보수는 매우 중요하다. 사용자 요구사항 분석, 요구사항 구현을 위한 설계, 실제구현, 테스트하는 과정을 거쳐 진행된다.

◆ 소프트웨어 품질관리

- 소프트웨어 품질은 소프트웨어 공학에서 중요하게 다루는 주제 중 하나로 크게 두 가지 유형으로 구분된다. 첫째는 산출 결과물인 제품에 대한 평가를 말하고, 둘째는 프로세스 품질에 대한 평가를 말한다.

◆ 소프트웨어 공학의 발전동향

- 최근 소프트웨어 공학은 웹 엔지니어링의 지속적 발전, 관점 지향 프로그래밍으로의 전환, 재사용을 위한 컴포넌트 수요의 증대 등으로 빠르게 진화하고 있다.